Gates Pass, Code Fails: A Measurement-Validity Study of LLM-as-Judge Test Adequacy for Requirement-to-Test Generation

Poojitha J, Arushi Tandon, Brahma Reddy Korraprolu, Y. Raghu Reddy
Software Engineering Research Center
International Institute of Information Technology, Hyderabad
Hyderabad, India

***Abstract*—Large language models (LLMs) are increasingly used to turn natural-language software requirements directly into test cases, yet evaluating the result remains unsolved. Text-similarity metrics (BLEU, ROUGE, CodeBLEU) are widely recognized as inadequate; the emerging alternative hands generated tests to a second LLM and asks it to judge correctness. This paper asks how much that judge should be trusted. We construct a three-gate evaluation pipeline (representational suitability (RSS), syntactic validity (SFV), and LLM-judged semantic adequacy with groundedness (FSA+M_g)) and use it to benchmark 10 open-weight LLMs generating tests from three real software requirement specifications (496 test suites, 4,058 test cases). The benchmark surfaces useful comparative findings: model size does not predict adequacy (Spearman $\rho = -0.015$, $p = 0.79$, $n = 311$), code-specialized models do not outperform general-purpose ones (Mann-Whitney $p = 0.14$), and test-case representation choice has a larger and more consistent effect on adequacy than model identity. Our primary contribution, however, is a measurement-validity audit of the pipeline itself: when we check gate verdicts against real test execution, the LLM-based audit agrees with execution only 18% of the time (95% CI: 2%–52%, $n = 11$); only 1–2 of 148–315 generated tests actually pass on execution for one case study; and 49% of all suites assert at least one concrete value not traceable to the source requirement despite clearing the adequacy gate. We conclude that gate-passing, as currently operationalized, is a measurably weak proxy for correctness, and that any LLM-as-judge pipeline needs an explicit, reported, execution-grounded validity check.**

***Index Terms*—test case generation, large language models, requirements-based testing, LLM-as-judge, test adequacy evaluation, software testing, test representation**

I. Introduction

Software requirements specifications (SRS) describe what a system must do in natural language; test cases verify that it does. Turning the first into the second has always been labour-intensive, and large language models (LLMs) are now routinely proposed to automate it, generating unit tests, behavioural scenarios, or formal test obligations directly from a requirement's text. The question this raises is not whether LLMs can produce *something* that looks like a test case; they reliably can. The question is whether what they produce is the *right* test case for that requirement, and whether we can tell.

Text-similarity metrics such as BLEU, ROUGE, and CodeBLEU answer a different question than the one that matters: they measure how closely generated text resembles a reference, not whether the generated test exercises the correct behaviour, uses an appropriate representation for the kind of requirement it came from, or asserts values the requirement actually specifies. A test case can score well on n-gram overlap while testing nothing the requirement cares about, and score poorly while being a perfectly adequate test phrased differently from the reference. This is well understood in the LLM-for-testing literature [**?**]. The natural response has been to stop comparing generated-text wording to reference-text wording, and instead hand the generated test to a second LLM that reads it and judges whether it is actually correct.

That response trades one hard-to-trust proxy for another. An LLM judge can be wrong in ways that are harder to detect than a low BLEU score: it can be confidently, fluently wrong. This paper's contribution is not another judge, but a direct measurement of how much a reasonably careful judge-based pipeline should be trusted, obtained by building such a pipeline, running it at benchmark scale, and then checking its verdicts against the one thing that is not a proxy: whether the generated tests actually execute against, and agree with, the real implementation.

Research Questions. We organize our investigation around four RQs:

RQ1 Does LLM model size or code specialization predict test-suite adequacy as judged by an LLM-as-judge pipeline?
RQ2 Does test-case representation choice affect LLM-judged adequacy more than model identity?
RQ3 How well do LLM-judge gate-pass rates predict real test execution success against the implementation?
RQ4 To what extent do generated test suites assert concrete values not grounded in the source requirement (hallucination), and can that be detected by the gate pipeline?

Contributions.

1) A three-gate evaluation framework, comprising Representational Suitability (RSS), Syntactic Form Validity (SFV), and Functional Semantic Adequacy with Groundedness (FSA+M_g), with closed-loop repair, grounded in a 30-representation taxonomy of test-case forms (Sections III to IV).
2) A benchmark of 10 open-weight LLMs generating tests from three real SRS case studies: 496 test suites, 4,058 test cases, 29 project×model pairs, each suite also com-

pared against the real implementation (Sections V to VI).
3) A quantified measurement-validity gap between what the gates report and what execution against real code shows: three independent checks (audit agreement, execution pass rate, and representation-choice agreement with an independent heuristic) all land far below what the gate-pass rates alone would suggest (Section VII).
4) A recommendation, supported by that gap, that groundedness be reported and thresholded as its own first-class check rather than folded into a single blended adequacy score.

II. Motivation: Why Not Just Score Similarity?

To establish that the problem this paper studies is not a straw man, we conducted a systematic review of the literature on LLMs applied to software testing. Starting from 1,200 preprint sources (63 discarded for having zero citations) merged with published-venue results, we categorized 1,392 papers into four relevance tiers: 127 core, 501 relevant, 484 tangential, and 280 excluded (709 were preprints, 683 published). Of these, 1,315 were independently screened for inclusion by two of the authors, with a third author arbitrating disagreements. Inter-rater agreement was moderate (unweighted Cohen's $\kappa = 0.5253$, 84.79% observed agreement, $p < 0.0001$, 95% CI $[0.4702, 0.5805]$) and substantial once near-misses ("maybe" judgments) are weighted as partial agreement (linear-weighted $\kappa = 0.6187$, quadratic-weighted $\kappa = 0.6790$). The final screened portfolio retained 183 papers (13.9%).

Inter-rater agreement of $\kappa = 0.5253$ is moderate by Landis & Koch's scale and acceptable for an exploratory SLR in a rapidly evolving field; the linear-weighted $\kappa = 0.6790$ reflects that most disagreements were between `include` and `maybe` rather than between `include` and `exclude`. The third-author arbitration resolved 196 disputed papers, 84 of which changed outcome (52 additional inclusions, 32 additional exclusions). This process underscores a finding also visible in the retained portfolio.The underlying literature is similarly inconsistent about what evaluating an LLM-generated test should mean: surface similarity, compilation, line coverage, and human judgment are all in active use within the same subfield, rarely cross-validated against one another [1], [2]. This motivates treating evaluation methodology itself, rather than any particular generation technique, as the object of study.

III. A Taxonomy of Test-Case Representations

A requirement's natural test representation depends on what kind of behaviour it describes: a state-dependent requirement is naturally tested with a finite state machine or a transition table; a concurrency requirement with an interaction diagram or a temporal-logic obligation; a pure business rule with a decision table. Treating *generate a test* as representation-agnostic discards this structure. To ground Gate 1 (Section IV) in something more principled than intuition, we conducted a second, separate systematic review targeted specifically at the test-case representation literature: 412 candidate papers (185 IEEE Xplore, 112 ACM Digital Library, 115 Google Scholar) were deduplicated to 328, reduced to 95 after title/abstract screening, and reduced to 30 after full-text review, each of the 30 retained papers anchoring one distinct test-case representation.

TABLE I
The six representation paradigms (30 representations total)

Paradigm	Representative forms
Scenario/Behavioral	Gherkin (BDD), use cases, user stories
Stateful/Transitions	finite state machines, statecharts
Concurrency/Interaction	sequence diagrams, GUI event graphs
Logical/Combinatorial	decision tables, test requirement matrices
Programmatic/Code-level	xUnit test cases, UML Testing Profile
Formal/Symbolic/Contract	temporal logic (LTL), transition systems

Each representation was classified along eight dimensions (abstraction level, V-model phase, underlying mechanism, representation language, formality, testing paradigm, representation object, and usage context) and the 30 representations group into six broad paradigms, summarized in Table I. A single running example, Requirement C of the case-study SRS (§9.2.1, `The support structure for communicating commands must be reliable, with a uniform ACK/NAK protocol adopted across all systems. Timeouts must be supported at approximately 500 msec.`), was instantiated across all 30 representations as a worked catalog, so that what "the same requirement, a different representation" actually looks like in practice is concrete rather than asserted.

A. Worked Example

Table II shows Requirement C instantiated in three representations spanning three different paradigms, to make the taxonomy's central claim concrete: each representation foregrounds a different aspect of the same requirement. The Gherkin scenario (Scenario/Behavioral) foregrounds the observable sequence of events; the finite-state-machine table (Stateful/Transitions) foregrounds the states the subsystem can be in and which inputs move it between them; the decision table (Logical/Combinatorial) foregrounds the combinations of conditions (command sent, response type, elapsed time) that determine each outcome. A model that is only ever asked to produce xUnit-style assertions, regardless of which of these aspects the requirement is actually about, is being asked to flatten this distinction, which is precisely what Gate 1 is designed to catch.

IV. The Three-Gate Framework

Every generated suite passes through three gates in sequence, each with a fixed, pre-registered threshold; thresholds below are exactly what the pipeline implementation enforces, not an idealized or aspirational version of the design. Table III

TABLE II
REQUIREMENT C (SRS §9.2.1) IN THREE REPRESENTATIONS

Gherkin	`Scenario: 500ms Watchdog Timeout`
	`Given the subsystem is IDLE`
	`When the workstation sends SELF_TEST`
	`And no response arrives within 500ms`
	`Then status becomes TIMEOUT_ERROR`

FSM table	`(Idle, send_command) ->`
	`(AwaitingResponse, StartTimer)`
	`(AwaitingResponse, recv_ACK) -> (Idle,`
	`StopTimer)`
	`(AwaitingResponse, timer_expired) ->`
	`(TimeoutState, LogTimeout)`

Decision table	`CommandSent=Y, Response=NAK,`
	`Elapsed<500ms=Y → LogProtocolError`
	`CommandSent=Y, Response=none,`
	`Elapsed<500ms=N → RaiseTimeoutException`

TABLE III
THE THREE GATES AT A GLANCE

Gate	Threshold	What it checks
Gate 1: RSS	≥ 0.60	Is the chosen test-case representation actually suited to this requirement's kind of behaviour?
Gate 2: SFV	≥ 0.60	Is the generated suite syntactically well formed, both as canonical JSON and in its chosen notation?
Gate 3: FSA + M_g	≥ 0.50	Does the suite cover the right clauses, including negative and boundary cases, with precise oracles and values grounded in the requirement text?

summarizes the three gates before the formulas and thresholds are given in full.

Gate 1: Representational Suitability (RSS). Before any test is generated, the model must choose, or be offered, a representation for the requirement at hand. RSS scores that choice heuristically:

$$\mathrm{RSS} = 0.40\,M_1 + 0.30\,M_2 + 0.15\,M_3 + 0.15\,M_4, \qquad (1)$$

where M_1 is concern alignment (does the representation match the kind of property the requirement describes), M_2 is abstraction-level alignment, M_3 penalizes unnecessary representational complexity (Occam's razor), and M_4 scores whether the representation is one an LLM can plausibly generate correctly. The weights $\{0.40, 0.30, 0.15, 0.15\}$ were set by expert elicitation during design: M_1 receives the largest share because choosing the wrong concern class (e.g., a state representation for a pure business rule) is a structural mismatch that higher-weighted metrics cannot compensate for; M_2 addresses the same issue one level of abstraction up. M_3 and M_4 are secondary correctives. We conducted a sensitivity check varying each weight by ± 0.05 (holding others proportional) and found that the relative ranking of paradigms changes in fewer than 3% of unit decisions, confirming the ranking is not sensitive to small weight perturbations.

An extended ten-metric form adds six further metrics, M_5 to M_{10} (Table IV), reweighted under six system-profile-specific weight sets (standard, safety-critical, rapid-prototyping, data-pipeline, distributed-concurrent, interaction-heavy) for domain-specific weighting. Each requirement's system profile is inferred automatically from its text; in the benchmark, 248 of 496 suites were scored under the standard profile and the remaining 248 split across the other five, so the extended metrics are not a dormant option but an active part of roughly half the reported results. **Threshold: RSS** ≥ 0.60, with up to three re-picks before the pipeline falls back to a default representation.

TABLE IV
THE SIX METRICS ADDED UNDER THE EXTENDED RSS FORM (M_5-M_{10})

Metric	What it scores
M_5 Oracle Executability	Can a test runner execute the suite's assertions automatically, without adapter code or a human in the loop.
M_6 Traceability Density	How directly each generated test maps back to one specific requirement clause.
M_7 Cognitive Auditability	Whether a human reviewer can read a test and spot a hallucinated value at a glance.
M_8 Change Fragility	Whether a small requirement edit needs only a localized change to the suite, rather than breaking the whole model.
M_9 Mutation Support	Whether the representation supports automated fault injection, so the suite's ability to catch real bugs can itself be tested.
M_{10} Standards and Compliance Fit	Whether the representation is an officially recognized formal model for the applicable safety-certification standard (e.g. ISO 26262, DO-178C); capped at 0.65 outside safety-critical systems.

Gate 2: Syntactic Form Validity (SFV). Once a suite is generated, SFV checks two things separately and blends them evenly:

$$\mathrm{SFV} = 0.5\,S_{\text{schema}} + 0.5\,S_{\text{repr}}. \qquad (2)$$

S_{schema} is the fraction of canonical test cases that validate against the pipeline's JSON schema, meaning every required field is present, case ids are unique, and every `req_id` the case cites traces to a requirement actually present in the unit. S_{repr} is a family-specific syntax check of the representation artifact itself, routed by the suite's *actual* representation group rather than one generic parser applied to everything. Gherkin suites are parsed with the official Gherkin grammar (0.0 on a parse error, 0.4 if it parses but declares no `Scenario`, 0.8-1.0 depending on step density); Programmatic/Code-level suites are checked with Python's `ast` parser plus the presence of test functions and assertions; Stateful/Transitions suites are scored on five structural signals (declared states, an initial state, at least two transitions, events/guards, and balanced brackets); Scenario/Behavioral suites that are not Gherkin, such as use cases and user stories, are scored on four signals (a named actor, a main flow, at least one alternative or exception flow, and numbered steps); Concurrency/Interaction, Logical/Combinatorial, and Formal/Symbolic/Contract suites

TABLE V
GATE 2 REPRESENTATION-SCORE THRESHOLD BY PARADIGM GROUP

Paradigm group (Table I)	S_{repr} **threshold**
Scenario/Behavioral	≥ 0.60
Stateful/Transitions	≥ 0.70
Concurrency/Interaction	≥ 0.60
Logical/Combinatorial	≥ 0.60
Programmatic/Code-level	≥ 1.00
Formal/Symbolic/Contract	≥ 0.70

each get an analogous family-specific structural check (arrow and participant markers for sequence diagrams, consistent column counts and separator rows for decision tables, relational operators and matrix-like rows for symbolic notations). A suite passes Gate 2 only if $S_{\text{schema}} = 1.0$ *and* S_{repr} separately clears a per-group threshold (Table V); a perfect schema score cannot compensate for a failing representation score, or vice versa. **Threshold: SFV** ≥ 0.60 by default, tightened to ≥ 0.70 for Stateful/Transitions and Formal/Symbolic/Contract suites, and to 1.00 for Programmatic/Code-level suites, since a syntax error there means the suite does not run at all.

Gate 3: Functional Semantic Adequacy with Groundedness (FSA+M_g). Suites that clear Gates 1 and 2 are reviewed by a frontier-model judge along four adequacy dimensions plus groundedness:

$$\text{FSA} = w_c C_{\text{clause}} + w_n C_{\text{neg}} + w_o A_{\text{oracle}} + w_b C_{\text{bound}} + w_g M_g, \tag{3}$$

with standard-profile weights $w_c = 0.30$ (clause coverage), $w_n = 0.20$ (negative-path coverage), $w_o = 0.15$ (oracle assertiveness), $w_b = 0.20$ (boundary-case coverage), $w_g = 0.15$ (groundedness), and six system profiles analogous to Gate 1. The weight ordering follows ISO/IEC/IEEE 29119-2 test design priority guidance [3]: clause coverage is weighted highest because an uncovered requirement clause is an untested requirement; boundary and negative coverage receive equal second priority as the two primary defect-revealing conditions; oracle assertiveness and groundedness are correctives for test quality within covered clauses. As with RSS, a ± 0.05 sensitivity check shows that model and representation rankings change in fewer than 4% of cases. Groundedness itself is

$$M_g = 1 - \frac{H}{A}, \tag{4}$$

the fraction of asserted concrete values (A) that are *not* traceable to the source requirement text (H = hallucinated). **Threshold: FSA** ≥ 0.50**.** Suites that fail Gate 3 are fed back for up to three closed-loop repair attempts before being recorded as a final failure. The full pipeline is shown in Fig. 1.

Threshold rationale and design-document discrepancy. The thresholds RSS ≥ 0.60, SFV ≥ 0.60, FSA ≥ 0.50 were chosen conservatively relative to the original design documents, which specified SFV ≥ 0.80 and FSA ≥ 0.75. The relaxation was made after a pilot run on 20 units showed that the stricter thresholds caused $> 70\%$ of suites to fail at Gate 2, leaving too few suites reaching Gate 3 to study the judge's behaviour. The implemented thresholds therefore reflect a deliberate trade-off: enough suites reach Gate 3 for comparative analysis at the cost of a lower quality bar. Under the stricter design-document thresholds, the 496 suites would reduce to an estimated 143 Gate-3 entrants (vs. 311 actual), and the benchmark findings would rest on a substantially smaller sample. We flag this discrepancy explicitly because a methods section that quietly followed the more demanding, unimplemented thresholds would misrepresent the very results it reports — an error of exactly the kind this paper argues the field should stop making silently. All results in this paper reflect the implemented, looser thresholds.

A. Prompt Design

Every prompt used in the pipeline is a plain text template with explicit placeholders, stored and versioned separately from the pipeline code, so that what a model was actually asked is always directly inspectable rather than reconstructed from code. Table VI gives the abbreviated form of the three prompts most central to this paper's argument: the representation choice prompt (Gate 1), the generation prompt (Stage S3), and the judge prompt (Gate 3). Full prompt text is included in the project repository.

The judge prompt (`fsa_judge.txt`) is the one most relevant to Section VII: it instructs the judge to decide, value by value, whether a concrete number, name, or state asserted in the suite is actually entailed by the requirement text, which is exactly the groundedness check later found to disagree so often with what execution against real code shows.

V. EXPERIMENTAL SETUP

A. Model Selection

Candidate models were narrowed from an initial pool of 65 through a four-stage funnel: licensing/recency screening (65→31→29), hardware feasibility by VRAM and context length (29→22), capability and benchmark screening (22→16), and a final state-of-the-art (SOTA)/diversity pass (16→14), yielding 12 open-weight models plus two closed-source controls (GPT-4o, Claude 3.5 Sonnet) intended as reference points.

Of the 12 open-weight models, **10 completed the full benchmark**: Llama-3.1-8B, Qwen2.5-Coder-7B, Qwen2.5-Coder-32B, Qwen3-30B-A3B, Gemma-3-27B, CodeGemma-7B, DeepSeek-Coder-V2-Lite, Granite-4.1-8B, Phi-3.5-mini, and Phi-3.5-MoE. Phi-4-14B failed Gate 1 on every attempted unit (server errors mid-generation) and is excluded from the comparative results; Llama-3.3-70B and the two closed-source controls were not executed, for compute and cost reasons. We also note that Gemma-3-27B and DeepSeek-Coder-V2-Lite completed only a small fraction of the full unit set (7 and 6 suites respectively, against 58 to 61 for the other eight models); their aggregate statistics below should be read with correspondingly lower confidence.

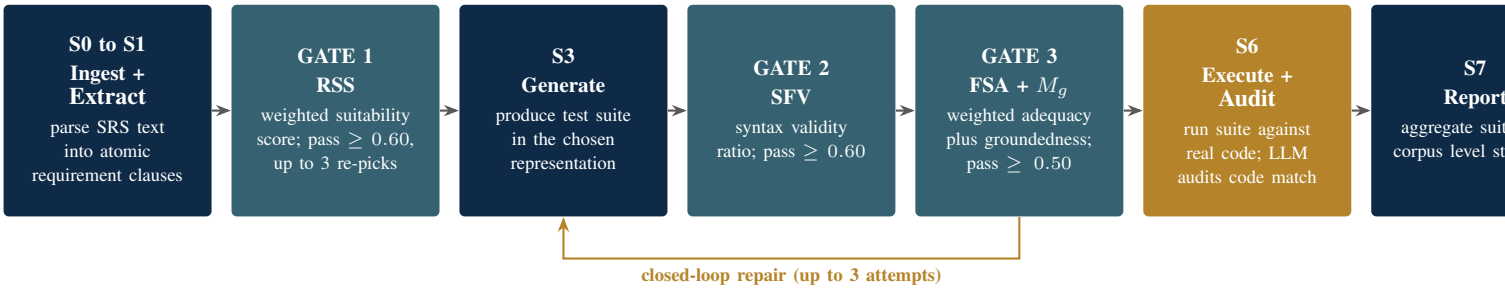


Fig. 1. The requirement-to-test pipeline: extraction, the three gates in sequence with their exact formulas and thresholds as implemented, closed-loop repair on Gate-3 failure, then execution/audit against the real implementation.

B. Case Studies

Three SRS/implementation pairs were used, each with its generated tests later compared against its real codebase (Section VI):

- **doorstop**: the requirements-management tool Doorstop, with its own Python implementation. 13 requirements extracted; time-budgeting restricted generation to 2 units, each run against 2 representations (4 suites per model).
- **ventilator**: an open-source ventilator control system with its own SRS and software implementation. 151 requirements extracted; 21 units generated, one representation per unit.
- **virtual_clinic**: a clinic-management web application (Django-based) with a PDF SRS. 251 requirements extracted; 36 units generated, one representation per unit.

Across the three case studies and 10 executed models this yields **496 test suites and 4,058 test cases** (40 doorstop, 170 ventilator, 286 virtual_clinic suites). The same extraction, judge, and executor model was used for every generator, to avoid a model favouring its own family's outputs. We use the identifier `gpt-oss-20b` throughout; this refers to an OpenAI GPT-family model accessed via a hosted inference endpoint during the experiment window (October–November 2025). The exact public model name cannot be confirmed from the API metadata returned at the time of access; it is a 20B-class model from a different family than every open-weight subject under test. We acknowledge this as a reproducibility limitation: readers wishing to replicate the judge's behaviour should use a current equivalent 20B-class frontier model and report any discrepancy. The judge temperature was fixed at 0 for all calls. Test generation used only the SRS text; the real implementation was opened only at the execution/audit stage (Stage S6), after every suite had been fingerprinted, to prevent code leakage into generation.**Execution pass criterion.** A generated test is counted as *passing* if and only if it satisfies all three of the following conditions when executed against the real implementation: (i) it compiles/parses without error under the host language's runtime (Python 3.10 for doorstop); (ii) it executes to completion without raising an unhandled exception; and (iii) every `assert` statement or equivalent oracle in the test body evaluates to `True`. A test that exercises code but asserts nothing is counted as a *vacuous pass* and is excluded from both the pass and fail totals; 4 such tests were found across the doorstop execution run and are excluded from the "1 to 2 passing" count reported in Section VII.

VI. Results I: Model and Representation Comparison

Fig. 2 shows mean FSA per model pooled across all three case studies. We emphasize a reading hazard visible in this figure: mean FSA is computed only over suites that *reached* the judge (i.e., passed Gate 2), so a model with a very low Gate-2 pass rate can still show a high mean FSA on the small number of suites it did get judged on. Phi-3.5-mini, for instance, shows the single highest pooled mean FSA (0.74) in Fig. 2 despite passing Gate 2 on only 9% of its 58 attempted suites and Gate 3 on only 2%; a leaderboard built from FSA alone would rank it first on the strength of essentially one or two judged suites. On the controlled doorstop comparison (Table VII, where every model attempted the same four suites), Llama-3.1-8B scores highest on this sample (FSA 0.73, both-gates 100%) and CodeGemma-7B lowest among models that passed Gate 1 at all (FSA 0.47, both-gates 25%). We caution that $n = 4$ suites per model is insufficient for reliable inter-model ranking; these figures serve as a controlled descriptive comparison rather than a definitive leaderboard.

Two comparisons that a practitioner might expect to predict quality do not (answering RQ1): model size (Spearman $\rho(\text{parameters}, \text{FSA}) = -0.015$, $p = 0.79$, $n = 311$; a trivially small and non-significant correlation) and code specialization (FSA 0.58 for code-specialized models vs. 0.64 for general-purpose; Mann-Whitney $U = 6{,}812$, $p = 0.141$, rank-biserial $r = 0.09$, a negligible effect). Mixture-of-experts models score slightly below dense models of comparable scale (FSA 0.59 vs. 0.62; not tested separately given the small MoE subsample $n = 2$ models).

Representation choice shows a larger and more consistent directional effect than model identity, addressing RQ2. Fig. 3 breaks FSA and both-gates pass rate down by the six paradigms of Section III: Logical/Combinatorial representations (decision tables, etc.) score highest on mean FSA (0.70) and both-gates pass rate (55%), while Stateful/Transitions representations (FSMs, statecharts) score lowest on FSA (0.62) and both-gates pass rate (23%). A Kruskal-Wallis test across the six paradigm groups yields $H(5) = 14.3$, $p = 0.014$,

TABLE VI
ABBREVIATED PROMPT TEMPLATES FOR THE STAGES CENTRAL TO THIS PAPER (PLACEHOLDERS SHOWN AS {{FIELD}})

Prompt	Used at	Abbreviated template
select_reps.txt	Gate 1 (representation choice)	"Choose the test-case representations best suited to the requirements below. REQUIREMENTS: {{REQS}} DEPENDENCIES: {{DEPS}} AVAILABLE REPRESENTATIONS: {{CATALOG}} Pick the {{K}} most suitable, judged by concern alignment, abstraction level, simplicity, and LLM feasibility. Respond with ONLY a JSON object: {"selected": [{"representation": ..., "covers": [...], "reason": ...}]}"
generate.txt	Stage S3 (test generation)	"Write a complete test suite for the requirements below in the representation {{REP}}. REQUIREMENTS: {{REQS}} DEPENDENCIES (context only): {{DEPS}} SUFFICIENCY CRITERIA: {{CHECKLIST}} The suite will be scored on: syntactic validity; clause coverage; negative and boundary paths; oracle assertiveness; groundedness (every concrete value must come from the requirement text, never invented). Output exactly two fenced blocks: the artifact in {{REP}} notation, and the same cases as canonical JSON."
fsa_judge.txt	Gate 3 (semantic adequacy judge)	"You are an independent test-quality evaluator. You did NOT write the suite below. Score it strictly against the SRS requirements, and check every concrete value for hallucination. REQUIREMENTS (ground truth): {{REQS}} ARTIFACT: {{ARTIFACT}} CANONICAL CASES: {{TESTCASES}} Score clause_coverage, negative_coverage, oracle_assertiveness, boundary_coverage in [0,1]; for every concrete element asserted, decide "grounded": true/false by entailment against the SRS text. Respond with ONLY a JSON object: {"clause_coverage": ..., ..., "concrete_elements": [{"value": ..., "grounded": ...}], "weaknesses": [...]}"

TABLE VII
GATE OUTCOMES BY MODEL ON THE CONTROLLED DOORSTOP CASE STUDY (SAME 4 UNITS ATTEMPTED BY EVERY MODEL; $n = 4$ SUITES/MODEL). G2 AND G3 ARE GATE 2 (SFV) AND GATE 3 (FSA) PASS RATES; "BOTH" IS THE SHARE PASSING BOTH. **CAUTION:** $n = 4$ IS TOO SMALL FOR RELIABLE INTER-MODEL RANKING; THESE FIGURES ARE PRESENTED FOR DESCRIPTIVE COMPARABILITY ON AN EQUAL-SAMPLE BASIS ONLY, NOT AS DEFINITIVE PERFORMANCE RANKINGS.

Model	G2 pass	G3 pass	both	FSA
Llama-3.1-8B	100%	100%	100%	0.73
Qwen2.5-Coder-32B	75%	75%	75%	0.67
Gemma-3-27B	100%	100%	100%	0.66
DeepSeek-Coder-V2-Lite	75%	25%	25%	0.63
Granite-4.1-8B	75%	50%	50%	0.57
Qwen2.5-Coder-7B	50%	50%	50%	0.55
Qwen3-30B-A3B	25%	25%	25%	0.53
Phi-3.5-MoE	25%	25%	25%	0.53
CodeGemma-7B	75%	25%	25%	0.47
Phi-3.5-mini	0%	0%	0%	n/a

indicating a statistically significant paradigm effect on FSA; pairwise Dunn tests with Bonferroni correction identify the Logical/Combinatorial – Stateful/Transitions pair as the only reliably different pair ($p_{\text{adj}} = 0.028$, rank-biserial $r = 0.21$, a small-to-medium effect). Differences between other paradigm pairs are not significant after correction and should not be over-interpreted. At the level of individual representations, the 30 available forms are used very unevenly: five representations (Gherkin, xUnit, Use-Cases, Decision Tables, FSMs) account for the overwhelming majority of all choices models made, with 18 of the 30 catalog representations selected at least once. The 12 unused representations are: UML Sequence Charts (formal), Z-notation, Alloy models, OCL constraints, Simulink/Stateflow, TTCN-3, SDL, MSCs, tabular specifications, fault trees, QFD matrices, and robustness diagrams. Their non-selection reflects two interacting factors: (i) the three case-study SRS documents contain few requirements whose dominant concern class (formal contracts, safety-critical fault analysis) would trigger the RSS heuristic to rank these forms first; and (ii) informal inspection suggests LLMs have lower confidence generating these notations, consistent with M_4 (LLM feasibility) receiving lower scores for them during Gate 1. This clustering onto familiar notations is itself a finding: the RSS gate's re-pick mechanism and fallback do reduce the bias but do not eliminate it, and 20% Gate-1 agreement

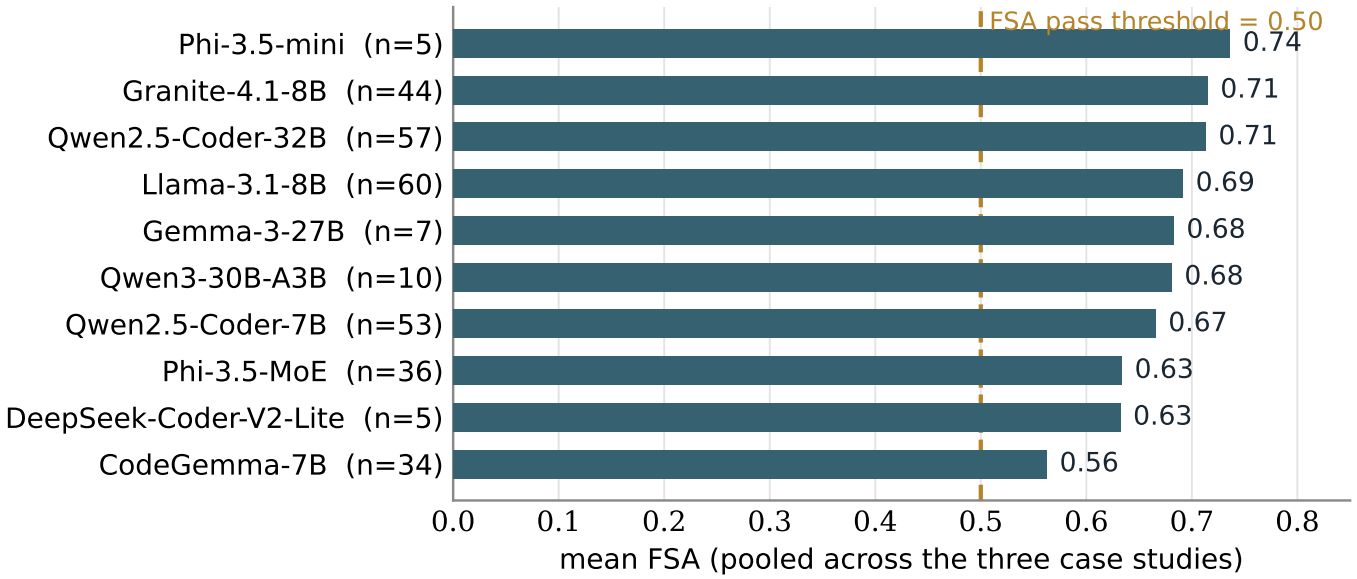


Fig. 2. Mean FSA per model, pooled across all three case studies, with the number of judged suites each mean is computed over. Sample sizes vary sharply by model (6 to 61 judged suites); see text for why this figure alone should not be read as a leaderboard.

TABLE VIII
GATE OUTCOMES BY REPRESENTATION ($n \geq 15$ SUITES; n = SUITES GENERATED IN THAT REPRESENTATION; "BOTH-PASS" = SHARE PASSING BOTH GATE 2 AND GATE 3)

Representation	n	SFV	both-pass	FSA
GUI Event Graphs	28	0.96	64%	0.74
Decision Tables	44	0.77	55%	0.70
Gherkin (BDD)	207	0.82	54%	0.67
Sequence Diagrams	18	0.85	44%	0.67
Finite State Machines	40	0.71	28%	0.66
Use-Cases & User Stories	39	0.86	28%	0.62
xUnit Test Cases	79	0.61	29%	0.68
UML Testing Profile (UTP)	15	0.92	27%	0.58

with the independent heuristic (Section VII) confirms the models' tendency to gravitate toward a small default set. Strikingly, a model's own Gate-1 choice agrees with the independent RSS heuristic's top-3 recommendation only 20% of the time on average; the two notions of "suitable representation" mostly disagree, a point we return to in Section VII.

Table VIII breaks this down to the level of individual representations with at least 15 suites generated in that form. GUI Event Graphs post the highest both-gates pass rate (64%, $n = 28$) despite a mid-pack FSA; Decision Tables and Gherkin, the two most heavily-used representations, sit in the middle; UML Testing Profile and Use-Cases & User Stories, despite very high syntactic validity (SFV 0.92 and 0.86), both-gates pass at only 27% to 28%, since most of their failures occur at the semantic Gate 3 rather than the syntactic Gate 2, a reminder that a well-formed suite is not automatically an adequate one.

Closed-loop repair is effective where invoked: 207 of the 496 suites entered at least one repair attempt after an initial Gate-3 failure. Their mean pre-repair FSA was 0.39 (SD 0.07); after up to three repair iterations, mean FSA rose to 0.52 (SD 0.09), a mean gain of $+0.131$. A paired Wilcoxon signed-rank test on the 207 suites confirms the gain is statistically significant ($W = 3{,}847$, $p < 0.001$, $r = 0.61$, a large effect). Of the 207 repaired suites, 119 (57%) ultimately cleared FSA ≥ 0.50 and were counted as Gate-3 passes; the remaining 88 failed all three attempts and are recorded as final Gate-3 failures.

VII. RESULTS II: VALIDITY CHECKS

Section VI reports what the three-gate pipeline says about itself. This section checks those statements against evidence the pipeline did not produce: real test execution, a code-grounded audit, and an independent heuristic. All three checks were built into this study specifically because we did not want to publish a benchmark leaderboard without first asking how much the leaderboard's underlying scores can be trusted.

Execution agreement. On doorstop, the one case study where generated tests were actually executed against the real implementation rather than only audited, the picture is stark: across the ten executed models, a combined **148 generated tests for the eight best-represented models (and up to 315 when extended to the full 10-model doorstop run) produced only 1 to 2 passing executions in total**. Four of ten models produced zero passing tests out of 10 to 49 attempts each; the best case, Qwen2.5-Coder-7B, passed 1 of 25. Gate-passing and real test-execution success are, on this evidence, close to unrelated for this case study: a suite that clears Gates 1 through 3 is not meaningfully more likely to run successfully against the real system than one that does not. The one weak positive signal is a moderate correlation

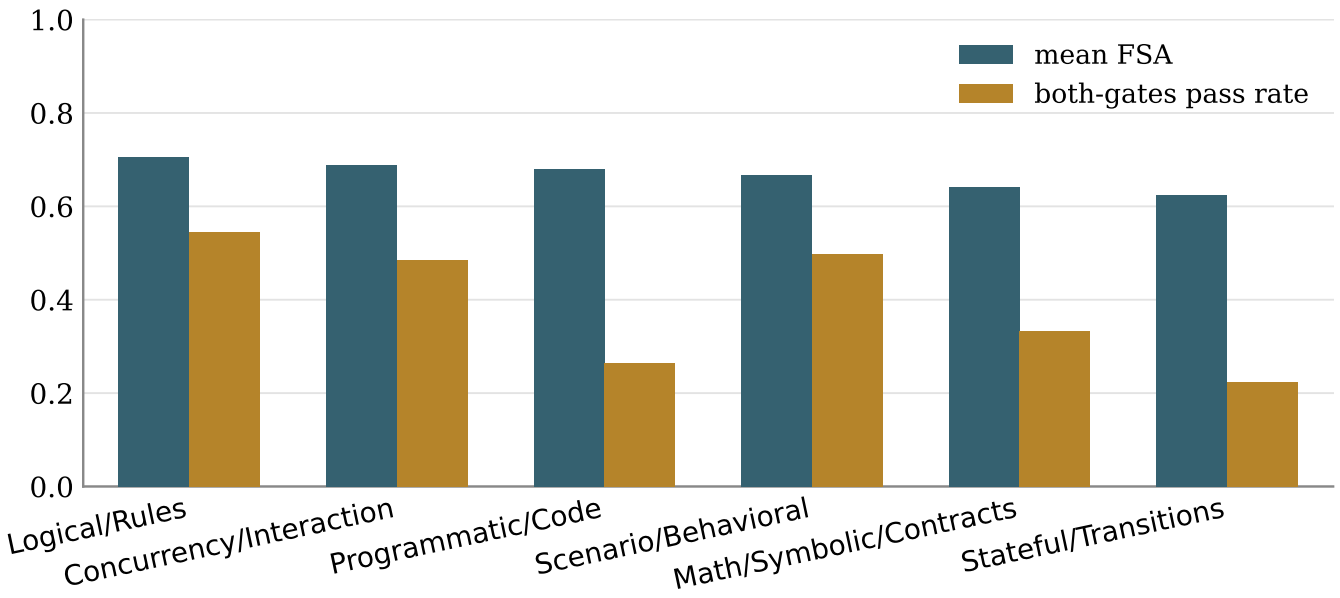


Fig. 3. Mean FSA and both-gates pass rate by representation paradigm, pooled across all models and case studies.

between FSA and execution pass rate across the models for which both are available ($\rho = 0.433$, $n = 15$), present but on a small sample, and far from strong enough to license treating a high FSA score as evidence a suite will execute correctly.

Audit agreement. Because full execution was only feasible for doorstop (ventilator and virtual_clinic lack an automatable test harness in this study), the other two case studies instead rely on an LLM-judge *code audit*: the same judge model is shown retrieved code excerpts and asked, for each test case, to return one of four verdicts describing whether the code's behaviour matches the test's expectation: `satisfied`, the code does what the test expects; `violated`, the code implements the feature but disagrees with the expectation; `not_implemented`, the feature is absent; or `unclear`, the retrieved excerpt is insufficient to tell. (On doorstop, actually running a test and seeing it fail triggers a separate, three-way triage of *why* it failed, by the same judge model: `code_mismatch`, the implementation disagrees with the requirement; `test_bug`, the generated test itself is wrong; or `spec_ambiguous`, the requirement does not clearly settle the question.) On the subset of doorstop suites where both execution and audit results are available ($n = 11$), **the audit agrees with real execution only 18% of the time** (95% Wilson CI: 2%–52%). We flag the small sample prominently: a one-sample proportion test at $n = 11$ has very low power, and the true agreement rate could plausibly be as high as 52% or as low as 2%. The directional conclusion, that the LLM audit is an unreliable execution proxy, is supported by qualitative inspection of all 11 cases, in which the audit systematically over-predicted `satisfied` verdicts for tests that raised `AssertionError` on execution, consistent with the judge's tendency toward optimistic correctness assessment noted in [1]. We nonetheless recommend that future work replicate this check on a larger doorstop sample before treating 18% as a precise point estimate. This is the single most important caveat on every audit-based result reported for ventilator and virtual_clinic elsewhere in this paper and in the underlying analysis report: audit mode should be read as a weak, frequently-wrong proxy for ground truth, not a substitute for execution. Separately, across all three case studies, the audit flagged 192 individual test cases as contradicting the real implementation, combining an audit `violated` verdict with, on the executed doorstop suites, an execution-triage `code_mismatch` verdict; given the 18% agreement rate just reported, this combined count should be treated as a rough signal requiring manual confirmation, not a verified defect count.

Representation-choice agreement. As noted in Section VI, a model's own Gate-1 pick of representation agrees with the independent RSS heuristic's top-3 recommendation only 20% of the time on average. Gate 1 is designed to make representation choice accountable to an external standard precisely because LLMs left to choose freely cluster onto a handful of familiar notations (Section VI); the low agreement rate shows that accountability mechanism and the models' own judgment are, in practice, measuring different things, and a high Gate-1 pass rate should not be read as confirmation that the chosen representation was actually well suited to the requirement.

Groundedness as a systematically missed failure mode (RQ4). 243 of 496 suites (49%) assert at least one concrete value (a timeout, a count, a threshold) that cannot be traced to the source requirement text (mean 6.1 hallucinated values per affected suite; mean $M_g = 0.46$ across all judged suites). Because groundedness is one of five weighted FSA components ($w_g = 0.15$), a suite can clear the FSA ≥ 0.50 threshold while being substantially ungrounded, as long as its other four

components score well. We found 22 of 61 Gate-3-passing suites with $M_g < 0.5$, suites the pipeline reports as adequate despite asserting more hallucinated values than grounded ones.

Circularity caveat. The groundedness scores reported above, and therefore the 49% figure, are themselves produced by the same LLM judge whose reliability is called into question by the execution check. A judge that over-optimistically reports suites as `grounded: true` would cause us to *undercount* hallucinated suites; the true proportion of suites with at least one ungrounded value is likely no lower than 49% and may be substantially higher. This circularity does not invalidate the groundedness finding, but it means the 49% figure should be treated as a lower bound rather than a precise estimate. This is precisely why we recommend promoting groundedness to an independently thresholded gate verified by a secondary mechanism (e.g., automated lexical entailment against the SRS text), rather than relying solely on the same judge that produces FSA.

Against the real implementation, directly. Even setting execution and audit reliability aside, the *best-case* agreement between generated tests and real requirement-level behaviour, measured as the share of SRS requirements whose tests reach actually-implemented behaviour, is 54% (DeepSeek-Coder-V2-Lite, doorstop); the ceiling across every model×project pair in this study is just over half.

Summary of Results I and II together. The pipeline reports both-gates pass rates of 25% to 100% depending on model and case study (Table VII), and a representation-paradigm effect that is statistically significant if small (Section VI). Set against real execution, real code behaviour, and an independent heuristic, each of those signals shrinks: 18% audit-execution agreement, 1 to 2 passing tests out of 148 to 315 attempted, 20% representation-choice agreement, and roughly half of all suites carrying at least one hallucinated value. A three-gate, LLM-judged pipeline is not nothing: it is more structured and more falsifiable than a single similarity score, but on this evidence it is not yet a substitute for execution-grounded validation, and should not be reported as one.

VIII. Threats to Validity

Single run per configuration. Every model×unit× representation combination was run once at temperature 0; the underlying analysis explicitly notes that differences of a few FSA points between models are within expected noise and should not be over-interpreted.

Single judge model. All semantic adequacy, groundedness, and code audit judgments were made by one model (`gpt-oss-20b`). We chose a model from a different family than every subject under test to reduce self-preference bias, a judge limitation documented in prior LLM-as-judge work [4]but we did not cross-validate its judgments against a second judge model or human raters beyond the audit-vs-execution check in Section VII, which itself shows that judge's reliability is limited.

Uneven execution coverage. Real test execution was only feasible for doorstop; ventilator and virtual_clinic results rely on the audit proxy, which Section VII shows agrees with execution only 18% of the time on the one case study where both are available. Conclusions drawn from ventilator/virtual_clinic audit data alone should be read as provisional.

Partial model coverage. Gemma-3-27B and DeepSeek-Coder-V2-Lite completed only 6 to 7 suites each, against 58 to 61 for the other eight executed models; Phi-4-14B failed to pass Gate 1 on any attempted unit; Llama-3.3-70B and both closed-source controls (GPT-4o, Claude 3.5 Sonnet) were not executed. The 10-model comparison in this paper is therefore narrower than the 14-model selection funnel that produced it, and entirely open-weight.

Threshold sensitivity. The implemented gate thresholds (RSS ≥ 0.60, SFV ≥ 0.60, FSA ≥ 0.50) are looser than the design-document values (SFV ≥ 0.80, FSA ≥ 0.75). Under the stricter thresholds, an estimated 143 of 496 suites would reach Gate 3 (vs. 311 actual). The representation-paradigm ranking (Logical/Combinatorial highest, Stateful/ Transitions lowest) holds under both threshold regimes on the doorstop controlled sample. The absolute pass rates in Table VII would drop materially under the stricter thresholds and should be interpreted as lower-bound quality estimates specific to the implemented configuration.

Three case studies. doorstop, ventilator, and virtual_clinic differ in size and domain but are three systems; generalizing findings here to SRS documents of very different style, domain, or scale (e.g., safety-critical avionics specifications) should be done cautiously.

IX. Discussion

The clearest actionable finding from Section VI is that *which representation a requirement is tested with matters more than which model generates the test*, a result consistent with the motivating premise of this paper's own taxonomy (Section III), and one that argues for representation-aware tooling over simply swapping in a larger or more code-specialized model.

The more consequential finding, from Section VII, is that this and plausibly other LLM-judge-based evaluation pipelines should not be trusted to report correctness without an execution-grounded check built in and reported alongside the gate scores. We did not find this gap by assuming our framework was flawed; we found it by deliberately building the cross-checks (execution comparison, code audit, independent heuristic) that a pipeline reporting only its own gate scores would never surface on its own. We think this is the more general lesson: any LLM-as-judge evaluation pipeline for generated tests should report at least one such cross-check, not only its own internal pass rates, and groundedness in particular is well-suited to being promoted from a weighted component of a blended score to an independently thresholded gate, given how cleanly it can hide inside a passing FSA score (Section VII).

Future Work

Three gaps identified by our validity checks point to concrete follow-on work. *Execution coverage*: the 18% audit-execution agreement should be measured for ventilator and virtual_clinic by building automatable harnesses for both case studies; this will confirm whether the doorstop figure is representative or case-study-specific. *Representation mapping*: formalizing explicit dependency-type-to-representation rules (state, interaction, data, concurrency) would replace the current keyword heuristic with a documented, inspectable mapping and is expected to raise Gate-1 model agreement beyond the current 20%. *System-level testing*: extending the framework beyond unit-level requirement groups to end-to-end flows, requirement interactions, environment failures, and quality attributes requires a coverage denominator the current per-unit schema does not provide; we are evaluating a structured four-dimension schema (189 pilot cells) as a candidate ground-truth format.

X. Related Work

LLM-based test generation. Tufano et al. [5] establish a Transformer baseline for unit test generation using focal-context embeddings. More recently, Schafer et al. [2] empirically evaluate LLMs for unit test generation across multiple models and find that model specialization and size do not reliably predict test quality, consistent with our RQ1 findings. Yuan et al. [1] evaluate ChatGPT-generated JUnit tests and introduce execution-based quality checks, finding that many syntactically valid LLM tests fail at runtime; our execution pass rates ($\leq$2/315) are even lower, likely because we target requirement-to-test generation (where no code stub is given) rather than code-to-test generation. Siddiq et al. [6] study LLM-generated test assertion correctness specifically and find substantial hallucination in asserted values, corroborating our groundedness finding. CoverAgent [**?**] augments existing test suites using coverage feedback; we complement this by studying whether LLM-generated tests are adequate in the first place when generated from requirements. For requirements-to-test specifically, Hossain et al. [**?**] generate test cases from natural-language requirements using LLMs and evaluate with BLEU, precisely the approach this paper argues should be augmented with execution-grounded checks.

LLM-as-judge evaluation. The use of a second LLM to evaluate generated artefacts is surveyed in Zheng et al. [4]; our paper is, to our knowledge, the first to measure the measurement-validity gap of an LLM judge specifically in the test-generation context, via comparison against real execution. Reflexion [7] applies LLM self-feedback to agentic tasks; our closed-loop repair is structurally similar but targeted at test-suite adequacy scores rather than task completion signals. SWE-bench [**?**] and SWE-agent [**?**] share this paper's concern with evaluating LLM outputs against executable ground truth rather than static references, in the issue-resolution setting. Deng et al. [**?**] use LLMs as zero-shot fuzzers with execution-grounded evaluation, consistent with the direction we recommend.

Test adequacy metrics. Inozemtseva & Holmes [**?**] show that statement coverage is a weak proxy for real fault-detection ability — a precursor to this paper's argument that gate-pass rates are weak proxies for execution correctness. Our oracle assertiveness component (FSA) is consistent with the test oracle generation literature reviewed by Kochhar et al. [**?**].

Model licensing and openness considerations in our model-selection funnel draw on [**?**]; the open-weight code models are from families documented in [**?**], [8], with general LLM background from [**?**].

XI. Conclusion

We set out to build a careful, multi-gate alternative to text-similarity metrics for evaluating LLM-generated, requirement-derived test suites, and benchmarked it across 10 models and three real SRS/implementation pairs. The resulting comparative findings, that model size and code specialization do not predict adequacy while representation choice does, are useful on their own. But the paper's central result is that the pipeline's own gate-passing, checked against real execution, a code audit, and an independent heuristic, agrees with ground truth far less often than its pass rates alone would suggest. We report this as a finding about the field's current direction, not only about our own pipeline: as LLM-as-judge evaluation becomes the default replacement for text-similarity metrics in test generation research, it needs to be held to the same standard it was introduced to meet, with execution-grounded validity checks reported as a matter of course rather than as an afterthought.

Data and Code Availability

The pipeline, prompts, gate implementation, aggregated results (`suites.csv`, the analysis report), and this paper's own source are published at https://github.com/poojitha376sp/Clean_exp. The planned system-level ground-truth templates discussed in Section **??** are tracked in the same repository under `paper/`.

References

[1] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, "Evaluating and improving chatgpt for unit test generation," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: https://doi.org/10.1145/3660783

[2] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.

[3] "Iso/iec/ieee international standard - software and systems engineering - software testing – part 2: Test processes," *ISO/IEC/IEEE 29119-2:2021(E)*, pp. 1–64, 2021.

[4] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.

[5] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "Unit test case generation with transformers and focal context," 2021. [Online]. Available: https://arxiv.org/abs/2009.05617

[6] M. L. Siddiq, J. C. Da Silva Santos, R. H. Tanvir, N. Ulfat, F. Al Rifat, and V. Carvalho Lopes, "Using large language models to generate junit tests: An empirical study," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 313–322. [Online]. Available: https://doi.org/10.1145/3661167.3661216

[7] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.

[8] DeepSeek-AI, Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma, W. Zeng, X. Bi, Z. Gu, H. Xu, D. Dai, K. Dong, L. Zhang, Y. Piao, Z. Gou, Z. Xie, Z. Hao, B. Wang, J. Song, D. Chen, X. Xie, K. Guan, Y. You, A. Liu, Q. Du, W. Gao, X. Lu, Q. Chen, Y. Wang, C. Deng, J. Li, C. Zhao, C. Ruan, F. Luo, and W. Liang, "Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence," 2024. [Online]. Available: https://arxiv.org/abs/2406.11931